



ASTRALIS UI

The Educational Report

How the library works, from the ground up — styling, tokens, theming, type safety, polymorphism, hooks, and the special implementation inside every component category.

7 July 2026

Read this before the Analysis Report.

Generated from a full source, build & runtime audit

1. The 30,000-foot picture

Astralis UI is a **styled, batteries-included React component library** — the same species as Chakra UI or Mantine, not the headless species (Radix, Ark) and not the copy-paste species (shadcn/ui). A consumer installs one package, imports one CSS file, wraps their app in one provider, and gets 50 themed components:

```
import "astralis-ui/styles.css";
import { AstralisProvider, Button } from "astralis-ui";

<AstralisProvider>
  <Button colorScheme="blue" size="lg">Launch</Button>
</AstralisProvider>
```

What actually ships to that consumer is just two artifacts in `dist/`:

- `astralis-ui.es.js` — all component logic, bundled by Vite (plus CJS/UMD builds and `index.d.ts` type declarations).
- `styles.css` — a single precompiled stylesheet containing *every class any component can ever emit*.

That second artifact is the defining architectural decision of the whole library, so let's start there.

2. The styling philosophy: Tailwind behind a curtain

2.1 What Tailwind is, in one paragraph

Tailwind is a compiler that scans your source code for class names like `p-4` or `bg-red-500` and generates exactly the CSS rules for the classes it found. In v4 it's configured *in CSS itself*: an `@theme { ... }` block declares design tokens, and each token namespace mints a family of utilities (`--color-x` → `bg-x`, `text-x`, `border-x` ...).

2.2 Why a library can't just use Tailwind naked

If Astralis shipped components with raw Tailwind classes, every consumer would need to:

1. **Run Tailwind themselves**, configured to scan `node_modules/astralis-ui` — fragile and slow.
2. **Match our Tailwind version and theme** — their `p-4` and our `p-4` would fight over one definition.
3. **See our styling implementation** — the library's internals become their problem.

2.3 The Astralis answer: prefix + precompile

Two lines solve all three problems. In `tailwind-entry.css`:

```
@import "tailwindcss" prefix(astralis);
```

- **prefix(astralis)** makes every utility ours: `p-4` becomes `astralis:p-4`. Our classes can never collide with a consumer's own Tailwind (or Bootstrap, or anything).
- **Precompiling** (`tailwindcss -m -i ./src/tailwind-entry.css -o ./dist/styles.css`) means the consumer never runs Tailwind. They get finished CSS, like they would from Chakra or MUI.

This is what a rendered Button actually looks like in the DOM (sampled live from the docs site):

```
class="astralis:inline-flex astralis:items-center astralis:justify-center
  astralis:font-medium astralis:cursor-pointer astralis:transition-all
  astralis:h-8 astralis:text-sm astralis:gap-1.5 astralis:rounded-lg
  astralis:bg-transparent astralis:text-accent-label
  astralis:hover:bg-accent-subtle astralis:accent-gray"
```

Note what's *not* there: no unprefixed Tailwind, no `tw-` anything, no hex colors. Honest caveat: a developer who knows Tailwind will still *recognize* the utility shapes (`inline-flex`, `h-8`) — the prefix prevents **collision and coupling**, not **recognition**. Truly opaque class names would require a hashing build step (like CSS Modules); that's a trade-off, not a defect.

2.4 The tax: every class must exist ahead of time

Precompilation has one iron law: **Tailwind can only generate classes it can see at build time**. A class name assembled at runtime (`"astralis:" + bp + ":" + cls`) is invisible to the scanner, so its CSS simply won't exist — and browsers ignore unknown classes *silently*. No error, no warning, just a style that doesn't apply.

This single fact explains three pieces of machinery you'll meet below: the **token maps** (§7), the **safelist generator** (§7.3), and the handful of `@source inline(...)` declarations in the entry CSS that force-generate classes used in unusual ways (like the ButtonGroup child selectors).

3. Design tokens: the three-layer pyramid

Everything visual in Astralis flows through CSS custom properties (a.k.a. CSS variables), organized in three layers. Understanding this pyramid is understanding the theming system.

Layer 1 — Primitives (raw ingredients)

`color.css` and friends define raw values with no opinion about usage:

```
--astralis-color-gray-100: oklch(...); /* just "a light gray" */
--astralis-color-red-500: ...;          /* just "a red" */
--astralis-spacing-4: 1rem;
```

Layer 2 — Semantics (roles, not colors)

`semantic.css` maps primitives onto **roles**. Components never say "gray-100"; they say "the subtle surface":

```
:root {
  --astralis-color-surface-base: var(--astralis-color-white);
  --astralis-color-surface-subtle: var(--astralis-color-gray-100);
  --astralis-color-label-base: var(--astralis-color-gray-900);
  --astralis-color-stroke-base: var(--astralis-color-gray-300);
}
```

There are two semantic vocabularies:

- **Global roles** (hueless): `surface-*` (backgrounds), `label-*` (text), `stroke-*` (borders), each with `base/muted/subtle/emphasized` intensities plus status variants (`warning/error/success/info`).
- **Per-palette roles**: every hue (red, blue, brand...) gets the *same* eight-word vocabulary — `solid` (filled bg), `contrast` (text on solid), `label` (tinted text), `subtle / muted / emphasized` (tint ramp), `stroke`, `ring` (focus). This uniformity is what lets one variant definition serve eleven color schemes (§5).

Dark mode is just Layer 2 re-declared. A `.astralis-dark` block assigns different primitives to the same roles (`surface-base` becomes near-black, `label-base` becomes near-white). Components don't contain a single dark-mode conditional — they reference roles, and the roles move. The class-based dark variant is declared once:

```
@custom-variant dark (&:where(.astralis-dark, .astralis-dark *));
```

so `astralis:dark:x` utilities key off a class on `<html>`, not the OS setting — which is what lets the user toggle themes in-app.

Layer 3 — The Tailwind bridge

The `@theme` block in `tailwind-entry.css` tells Tailwind to mint utilities *from our tokens*:

```
@theme {  
  --color-surface-base: var(--astralis-color-surface-base);  
  --color-accent-solid: var(--astralis-color-accent-solid);  
  --spacing-4: var(--astralis-spacing-4);  
}
```

Now `astralis:bg-surface-base` exists as a class, and its generated CSS is `background-color: var(--astralis-color-surface-base)`. **The `var()` indirection is the entire point**: because the compiled CSS still references the variable rather than a baked hex value, changing the variable at runtime (dark mode, brand override, accent scope) recolors already-rendered components without recompiling anything.

One deliberate exception, learned the hard way: **breakpoints are literal** (`--breakpoint-md: 48rem`, never `var(...)`), because CSS forbids `var()` inside `@media` queries — browsers would silently drop every responsive rule.

4. Runtime theming: the provider

`provider.tsx` (`AstralisProvider`) does four jobs:

1. **Theme state machine.** Holds `"light" | "dark" | "system"`, persists explicit choices to `localStorage`, resolves `"system"` via `matchMedia("(prefers-color-scheme: dark)")`, and subscribes to OS changes. To avoid SSR hydration mismatches it renders with server-safe defaults first and reads `localStorage` in an effect after mount.
2. **Applies the theme** by toggling `astralis-dark` on `<html>` — the class the `dark` variant and the `.astralis-dark` token re-declarations both key off.
3. **Scopes the reset.** The provider renders `<div class="astralis">`, and the entry CSS attaches box-sizing, font, background and border resets to `.astralis` and its subtree — so Astralis normalizes *its* island without bulldozing the host page's styles.
4. **Brand color injection.** `tokens.brandColor` feeds `generateBrandShades()`:

```
// hex in → 10 shades out, injected as inline style variables
const s100 = blend(0.16, 255, 255, 255); // tints: blend toward white
const s700 = blend(0.65, 0, 0, 0);        // shades: blend toward black
return { "--astralis-color-brand-50": ..., "--astralis-color-brand-900": ... };
```

One brand hex becomes a full 50–900 scale at runtime, by linear RGB blending toward white (tints) and black (shades). Because brand *utilities* reference `var(--astralis-color-brand-*)`, the inline variables win the cascade and everything branded recolors instantly.

The sharp edge to know about (it resurfaces prominently in the analysis): when a custom property's value contains `var()`, the substitution happens **at the element where the property is declared**, and descendants inherit the *already-substituted* result. `--astralis-color-brand-solid: var(--astralis-color-brand-500)` is declared at `:root`, so it is "baked" to the default yellow **at the root** — overriding `brand-500` lower down (which is exactly what the provider's inline styles do, on its own `<div>`) does **not** re-run that substitution. Components that consume *role* tokens (`bg-brand-solid`, the whole accent channel) keep the baked yellow; only utilities referencing primitive shades directly (`bg-brand-300`) see the override. The docs-site hero proved this empirically: its brand picker only became correct after manually re-declaring the role tokens (per theme) alongside the shades. The provider's `tokens.brandColor` does *not* do that re-declaration — a real bug catalogued in the analysis report.

5. The accent channel: how `colorScheme` costs almost nothing

The naive way to support `colorScheme` is to generate classes per hue per variant per property: `bg-red-solid`, `bg-blue-solid` ... for every component. CSS explodes multiplicatively.

Astralis instead defines a **virtual palette** called `accent`. Components paint exclusively with it:

```
solid: "astralis:bg-accent-solid astralis:text-accent-contrast ..."
```

and a tiny scope class rebinds what "accent" *means* for a subtree (`accent.css`):

```
.astralis-accent-red {  
  --astralis-color-accent-solid: var(--astralis-color-red-solid);  
  --astralis-color-accent-label: var(--astralis-color-red-label);  
  /* ...all eight roles */  
}
```

So `colorScheme="red"` compiles to *one extra class* (`color-schemes.ts`):

```
export const accentClass = (scheme: ColorScheme) => `astralis-accent-${scheme}`;
```

You can see it in the live DOM sample in §2.3: the button's styling classes are hue-agnostic (`bg-accent-subtle`), and the trailing `astralis-accent-gray` is the entire color scheme. Because each rebind points at the hue's *role* tokens — which already flip between light and dark values — the accent channel is automatically theme-aware. Adding a new scheme library-wide = one entry in `COLOR_SCHEMES` + one block in `accent.css`. That's $O(1)$ instead of $O(\text{hues} \times \text{variants} \times \text{components})$.

6. How a single component is styled: CVA + merge

6.1 CVA (class-variance-authority)

CVA is a small utility that turns a *variant table* into a class-string function, and — crucially — into **TypeScript types**. Button's structural layer:

```
const buttonVariants = cva(
  "astralis:inline-flex astralis:items-center ...", // base, always applied
  {
    variants: {
      size: {
        sm: "astralis:h-8 astralis:text-sm astralis:gap-1.5 astralis:px-3",
        md: "astralis:h-10 astralis:text-base astralis:gap-2 astralis:px-4",
      },
      rounded: { md: "astralis:rounded-md", full: "astralis:rounded-full" },
    },
    compoundVariants: [ /* rules that fire on combinations, e.g. iconOnly+size */ ],
    defaultVariants: { size: "md", rounded: "lg" },
  },
);
```

`buttonVariants({ size: "sm" })` returns the composed string; `VariantProps<typeof buttonVariants>` gives the exact prop union (`size?: "xs" | "sm" | ...`) so **autocomplete and type errors come from the style table itself** — there is no second, hand-maintained type to drift out of sync.

Button actually uses **two CVA layers**: the structural one above, and a color layer keyed only on `variant` whose values use accent tokens — composed with `accentClass(colorScheme)` at render. Structure and color are independent axes, which is exactly why every variant works with every scheme.

6.2 astralisMerge: last-one-wins classNames

Consumers pass `className` overrides. In plain CSS, `astralis:p-4 astralis:p-2` is a specificity coin-flip (source order wins, not author intent). `astralis-merge.ts` wraps **tailwind-merge** configured with our prefix:

```
const twMerge = extendTailwindMerge({ prefix: "astralis", extend: { classGroups: {
  "border-w": [{ border: ["normal", "moderate", "thick", ...] }], // teach it our custom
utilities
}}});
export const astralisMerge = (...inputs: ClassValue[]) => twMerge(clsx(inputs));
```

It understands which utilities *conflict* (two paddings) versus *coexist* (padding + margin) and keeps only the last of each conflicting group — so `<Button className="astralis:px-8">` genuinely overrides the built-in padding. Custom utilities Tailwind-merge doesn't know natively (like our named border widths) are registered in `classGroups`, otherwise it would treat `border-normal` and `border-stroke-base` as the same "border" group and wrongly drop one.

7. Responsive style props: the engine

7.1 The developer experience

Chakra-style per-breakpoint values on style props:

```
<Box p="4" />                → astralis:p-4
<Box p={{ base: "2", md: "4" }} /> → astralis:p-2 astralis:md:p-4
```

7.2 How it works: token maps + resolveStyleProps

Runtime string-building is forbidden territory (§2.4) unless every possible output exists in the compiled CSS. So the system is built on **token maps** — plain objects in `src/const/` that enumerate the complete vocabulary:

```
export const pSpacing = { "2": "astralis:p-2", "4": "astralis:p-4", ... } as const;
```

These maps are the **single source of truth three consumers share**:

1. **CVA** uses them as its `variants` (so scalar values and `defaultVariants` work, and types flow out).
2. **The responsive engine** looks up breakpoint overrides in them.
3. **The safelist generator** (next section) reads them at build time.

`resolveStyleProps` splits a prop bag: scalars go through CVA untouched; responsive objects are peeled apart, each breakpoint value resolved from the map and prefixed by `withBreakpoint`, which splices the breakpoint *inside* the prefix (`astralis:p-4` + `md` → `astralis:md:p-4` — the only shape Tailwind's prefixed variant system accepts).

7.3 The safelist generator: closing the loop

`astralis:md:p-4` is assembled at runtime → the scanner can't see it → its CSS wouldn't exist. `generate-responsive-safelist.mjs` runs before every CSS build: it regex-extracts every `astralis:*` literal from the token maps and component styles and emits

```
@source inline("astralis:{sm,md,lg,xl}:p-4");
```

for each — `@source inline` being Tailwind v4's "generate this even though you didn't see it used" mechanism (the brace expands to all four breakpoints). This is a **build-order coupling**: token maps → safelist → Tailwind compile. Change a map and skip the regeneration, and responsive variants silently rot. (The analysis report has findings about this exact regex.)

8. Type safety and autocomplete: where it all comes from

Three mechanisms, layered:

1. **as const maps** → **literal unions**. `keyof typeof pSpacing` is `"0.5" | "1" | ...`, not `string`. This is why your editor lists every valid `p` value: the type is the map.
2. **CVA `VariantProps`** extracts prop types from the variant tables, so `variant`, `size`, `colorScheme` autocomplete directly from the styles file.
3. **The `Responsive<T>` wrapper** (`responsive.ts`) lifts each variant prop to "scalar or per-breakpoint map":

```
export type ResponsiveProp<V> = V | Partial<Record<"base" | "sm" | "md" | "lg" | "xl", V>>;  
export type Responsive<T> = { [K in keyof T]: ResponsiveProp<NonNullable<T[K]>> };
```

So `p={{ base: "2", md: "4" }}` typechecks *and* `p={{ md: "banana" }}` errors — the breakpoint object is validated against the same union as the scalar. The tiny `Prettify<T>` helper in `types/index.ts` flattens these intersected mapped types so hover-tooltips show a clean object instead of type soup.

9. Polymorphism: the `as` prop

Components render as different elements while keeping type safety: `<Button as={Link} href="/docs">` must accept `href`; `<Box as="section">` must not.

Box is the canonical implementation:

```
type BoxComponent = <T extends ElementType = "div">({
  props: BoxProps<T> & { ref?: Ref<any> },
}) => ReactNode;

const Box = forwardRef(<T extends ElementType = "div">({ as, ... }, ref) => {
  const Element = (as || "div") as ElementType;
  // split props: token-map keys → style engine, everything else → the element
  for (const key in props)
    (BOX_VARIANT_KEYS.includes(key) ? variantProps : htmlProps)[key] = props[key];
  return <Element ref={ref} className={astralisMerge(resolveStyleProps(...), className)}
    {...htmlProps} />;
}) as unknown as BoxComponent;
```

Three ideas worth understanding:

- **The generic:** `BoxProps<T>` = our style props + `ComponentPropsWithoutRef<T>` (the native props of whatever `T` is). Pass `as="a"` and `href` appears in the type; pass `as="div"` and it doesn't.
- **The prop split** is *data-driven*: `BOX_VARIANT_KEYS = Object.keys(boxVariantMap)` — the runtime split is derived from the same map that defines the styles, so a new style prop can never leak onto the DOM element as an invalid attribute.
- **The cast** (`as unknown as BoxComponent`): `forwardRef`'s type signature can't express "a component that is itself generic," so every polymorphic library (Chakra included) erases and re-asserts the type. The runtime is fine; the cost is that the cast is a place where types can lie — noted in the analysis.

Button adds a semantic wrinkle: when `as` is a real `<button>` it uses the native `disabled` attribute, but when polymorphic (a link can't be `disabled`) it swaps to `aria-disabled + tabIndex={-1}` — same visual, correct semantics per element.

10. Compound components and context

Multi-part components expose their anatomy as dot-parts instead of a prop explosion:

```
<Card variant="elevated">
  <Card.Header extra={<Badge>Live</Badge>}>
    <Card.Title>Mission control</Card.Title>
  </Card.Header>
  <Card.Body>...</Card.Body>
</Card>
```

Mechanics:

- **Attachment** is plain `Object.assign(CardRoot, { Header, Title, ... })` — no magic, which is also why dot-access from a React Server Component fails (the server sees only the client reference, not its properties; docs demos using compounds are `"use client"` for this reason).
- **Context carries cross-part state.** `Card.Root` provides `size` so every section pads consistently; `Accordion.Root` provides `isOpen(value) / toggle(value)`; `ButtonGroup` provides `size/variant/colorScheme/disabled` so children inherit group settings unless they override. Two conventions coexist in the codebase — contexts that **throw** when a part is used outside its root (`DataList`, `Table`: loud and safe) and contexts with **defaults** (`Card`: silently renders md) — a consistency point the analysis flags.
- **ARIA wiring uses `useId()`**: `Accordion` generates stable ids to link `aria-controls` (trigger → panel) and `aria-labelledby` (panel → trigger); `Field` generates the id that ties `<label htmlFor>` to the input it wraps. React 18+'s `useId` is SSR-safe — server and client generate the same id, so hydration doesn't tear.

11. The hooks toolbox (the library's "engine room")

Five shared hooks in `src/hooks/` power all interactive behavior. Astralis wrote these itself instead of depending on Radix/Floating-UI — total control and zero dependencies, at the cost of owning every edge case.

`useControllableState` — the controlled/uncontrolled pattern

Every stateful component (Tabs, Modal, Accordion, Checkbox...) supports both usage modes: *uncontrolled* ("manage yourself, seed with `defaultValue`") and *controlled* ("I own the state, here's `value` + `onChange`"). The hook collapses that into one line per component:

```
const [uncontrolled, setUncontrolled] = useState(defaultValue);
const isControlled = value !== undefined;
const state = isControlled ? value : uncontrolled;
const setState = (next) => { if (!isControlled) setUncontrolled(next); onChange?.(next); };
```

If `value` is provided, internal writes are skipped and the parent is merely *notified* — the component becomes a pure function of its props. The naming convention is uniform across the library (`open/defaultOpen/onOpenChange`, `value/defaultValue/onValueChange`, `page/defaultPage/onPageChange`) — one of the library's quiet consistency wins.

`usePresence` — exit animations without a framework

CSS can animate an element *appearing*, but not one React has already unmounted. The hook keeps the node mounted through the exit:

- **Open:** mount immediately (so positioners can measure), then flip state to `"open"` after **two** `requestAnimationFrame`s — the double-frame guarantees the browser has painted the "closed" styles first, so the transition actually runs instead of jumping.
- **Close:** flip to `"closed"` (exit styles apply, transition plays), unmount after `duration` ms.

Modal (200ms), Drawer (250ms), Popover (160ms), Tooltip (140ms) all ride this — durations deliberately scaled to the size of the thing moving.

`useAnchorPosition` — a mini Floating-UI

Positions Popover/Tooltip/Select panels next to their trigger using `position: fixed` + viewport math: measure both rects → **flip** to the opposite side if the preferred side lacks room → **shift** (clamp) into the viewport → compute an **arrow offset** centered on the anchor. Recomputes on scroll (capture phase, so it hears scrolls inside nested containers) and resize. ~100 lines standing in for a 20KB dependency.

useDismiss vs useOverlayBehavior — two grades of "closeable"

- **useDismiss** (light dismiss, for non-modal Popover/Tooltip): Escape + pointer-down outside a set of refs. Capture-phase `pointerdown` so it beats inner handlers. No focus trapping — you can Tab away from a popover, by design.
- **useOverlayBehavior** (modal discipline, for Modal/Drawer): body scroll lock, deferred initial focus into the panel, Tab/Shift-Tab **focus trap** (cycles first ↔ last focusable), Escape to close, and **focus return** to whatever was focused before opening — the detail keyboard users notice most when it's missing.

Portal

Overlays render into `document.body` via `createPortal`, escaping any `overflow: hidden` ancestor. The wrapper renders `null` until after mount (`useEffect`-gated), which makes it SSR-safe: the server renders nothing, the client fills it in.

12. Category tour: the special implementation in each component

Buttons

- **Button**: two-layer CVA (§6.1); icon-only detection (`!children && (leftIcon || rightIcon || loading)` → square geometry); native-vs-polymorphic disabled semantics (§9); loading state swaps in a `currentColor` spinner with `loaderPlacement`.
- **ButtonGroup**: context pushes shared `size/variant/colorScheme/disabled` down; `attached` mode needs no child cooperation — pure CSS child selectors collapse inner radii and overlap borders:

```
"astralis:[&>*:not(:first-child)]:rounded-s-none astralis:[&>*:not(:last-child)]:rounded-e-  
none astralis:[&>*:not(:first-child)]:-ms-px"
```

(These arbitrary-selector utilities can't be extracted from `.ts` files by the scanner — they're force-generated by `@source inline` lines in the entry CSS.)

- **ThemeToggle**: a Button preset wired to `useTheme()`; sun/moon icons stacked in one frame, cross-faded with opacity+rotation.

Layout

- **Box** is the foundation (§9): ~130 style props across spacing/sizing/color/positioning/borders/shadows, all responsive.
- **Flex/Grid** extend Box with their axis vocabularies and ship `Flex.Item` / `Grid.Item` for per-child placement (`colSpan`, `order`, `alignSelf`). Grid adds raw-string escape hatches (`templateColumns`, `templateAreas`) applied as inline styles — the sanctioned bypass for arbitrary values that can't precompile.
- **Stack/HStack/VStack** are ergonomic Flex presets. **Center** is one-job Flex. **Container** = `w-full mx-auto` + default `maxW="5xl"` + gutters.
- **AspectRatio** locks the ratio and stretches its child via `[&>*:not(style)]:absolute inset-0 size-full` child selectors. **Float** anchors a badge at one of nine placements, each token expanding to a multi-class position+translate combo. **Separator** is semantic (`role="separator"` + `aria-orientation`).

Typography

- **Text**: every typographic prop responsive; element-aware defaults (`as="h1"` implies 4xl/bold before you say anything). **Heading** delegates to Text with heading font/tightened metrics.
- **Highlight**: safe substring marking — escapes regex metacharacters in the query, builds one union regex *with a capturing group* (so `String.split` **keeps** the delimiters), then alternates text / `<mark>`:

```
const regex = new RegExp(`(${terms.map(escapeRegex).join("|")})`, ignoreCase ? "gi" : "g");
const segments = children.split(regex); // odd indexes are the matches
```

- **List** = compound `List` / `List.Item`; an `icon` on an item suppresses the native marker and inlines the icon. **CodeBlock** renders a window-chrome header (traffic lights, language label) over a `<pre><code>` body and accepts pre-highlighted HTML — highlighting stays out of the bundle.

Data entry

- **Field** is the form glue: provides `id/invalid/disabled/required/readOnly` context that inputs consume, and `useFieldContext()` returns `null` outside a Field so every input also works standalone.
- **Checkbox/Radio/Switch** share the **sr-only native input + peer styling** pattern: a real `<input>` (keyboard, forms, screen readers — free) visually hidden, with the drawn control styled via peer state. All are group-aware: inside `Checkbox.Group` / `Radio.Group`, state defers to group context.
- **Select/MultiSelect**: trigger + portaled listbox positioned by `useAnchorPosition`; ARIA combobox pattern (`role="combobox"` / `listbox` / `option` + `aria-activedescendant` -style index tracking); search filtering; MultiSelect renders chips inline and Backspace-on-empty-search removes the last one.
- **Slider**: drag math converts pointer position → track fraction → stepped value; document-level move/up listeners read current props through a **ref** — the standard cure for stale closures in long-lived listeners:

```
const paramsRef = useRef({ min, max, step }); // listeners read .current, never a stale snapshot
```

- **PinInput**: one `<input>` per cell; typing auto-advances, Backspace retreats, and **paste** is intercepted, filtered through the type regex, and distributed across cells with focus landing at the end.

Disclosure

- **Accordion**: the height-animation trick — animating `height: auto` is impossible in CSS, so it animates `grid-template-rows: 0fr → 1fr` on a grid wrapper and lets the inner row clamp itself. Zero JS measurement:

```
<div style={{ gridTemplateRows: open ? "1fr" : "0fr" }} className="astralis:transition-all">
  <div className="astralis:overflow-hidden astralis:min-h-0" inert={!open}>{children}</div>
</div>
```

(`inert` also removes closed content from tab order and the accessibility tree.) Full roving arrow-key navigation across triggers.

- **Carousel**: no scroll-snap — a flex track moved by a computed transform, `calc(-1 * index * (100% + gap) / slidesPerView)`; `fade` mode stacks slides in one grid cell and cross-fades; autoplay, pause-on-hover, swipe, and proper `aria-roledescription="carousel"` semantics.

- **Pagination:** MUI-style windowing — first `boundaryCount` pages + last `boundaryCount` + `siblings` around the current page, with ellipses only where gaps genuinely exist.

Navigation

- **Tabs:** textbook roving tabindex (only the active tab is tabbable; arrows move focus; `activationMode` decides whether focus selects). The animated ink-bar indicator is a `ResizeObserver` measuring the active trigger and transitioning `left/width` (or `top/height` vertically).
- **Steps:** status derived, not stored — `index < step ? "completed" : index === step ? "active" : "upcoming"`; `linear` mode disables future steps; `Steps.Completed` shows once `step >= count`.

Overlay

Modal, Drawer, Popover, Tooltip = the hooks toolbox composed in different grades (§11): Modal/Drawer take `useOverlayBehavior` + Portal + `usePresence`; Popover swaps in `useDismiss` + `useAnchorPosition` (non-modal); Tooltip adds a show-delay timer, shows instantly on keyboard focus, and is `pointer-events: none` so it never steals the hover it depends on. Drawer's panel dimensions are applied as **inline styles** (`drawerSizeStyle()`), sidestepping class generation for size variants.

Data display

- **Avatar:** initials from name; deterministic fallback color by hashing the name into the scheme palette (same person = same color, every render):

```
for (let i = 0; i < name.length; i++) hash = name.charCodeAt(i) + ((hash << 5) - hash);
return PALETTE[Math.abs(hash) % PALETTE.length];
```

`Avatar.Group` overlaps via negative margin and renders a "+N" chip past `max`.

- **Calendar:** a real date engine — month grids honoring `firstDayOfWeek`, outside-day rendering, single/multiple/range selection with `data-selected` / `data-in-range` attributes, `Intl.DateTimeFormat` for locale-aware month/weekday names, min/max/unavailable constraints.
- **Table:** semantic HTML (`th scope="col"`, caption placement) with size/variant context; striping via `even:` variant; `stickyHeader` via sticky positioning.
- **Marquee:** children rendered **twice** in a flex track animated `translateX(0 → -50%)` — the second copy slides in exactly as the first exits, so the loop is seamless; duration derived from measured content width ÷ `speed` (px/s) so perceived speed is content-independent; keyframes injected once at runtime.
- **QRCode:** async SVG generation from the `qrcode` package, status overlays (loading/expired/scanned) that blur the code, optional center overlay and SVG download.
- **Timeline:** indicator column width matched to indicator size so the connector line threads through center; last-item context hides the trailing connector. **Stat/DataList/Tag/Badge** are disciplined token-consumers; `Tag.Checkable` implements `role="checkbox"` + Space/Enter for filter-chip UX.

Icon

Dual-mode: `as={LucideIcon}` renders the component with sized SVG attrs; raw `<svg>` children get a sized `<span>` wrapper with `[&>svg]:size-full`. Decorative by default (`aria-hidden`), switches to `role="img"` when you pass `aria-label`.

13. The build pipeline, end to end

```
src/**/*.tsx —vite build→ dist/astralis-ui.{es,cjs,umd}.js
|
|   └vite-plugin-dts→ dist/index.d.ts
|
| └scripts/gen-responsive-safelist→ src/_responsive-safelist.css
|   └tailwindcss -m -i tailwind-entry→ dist/styles.css (tokens + utilities + safelist)
```

- Vite bundles the library with React externalized (it's a peer dependency — the consumer's React is used, ours is never duplicated).
- The single `"use client"` at the top of `index.ts` is hoisted to the top of the bundle, marking the **whole library** as client code for React Server Component environments — Next.js apps can import components anywhere, and they'll render as client components (this blanket approach is discussed in the analysis).
- The docs site consumes `dist/` (not `src/`), which is why library changes need `pnpm build:ui` before they appear — the docs experience is the *consumer's* experience by construction.
- Storybook runs the same CSS pipeline into `src/index.css` for isolated component development.

14. The design values, in one list

If you strip away the implementation, the library keeps making the same five bets:

1. **Tokens over values** — components reference roles; themes move the roles.
2. **Indirection over duplication** — one accent channel instead of per-hue classes; one dark-mode block instead of per-component conditionals.
3. **Single source of truth** — token maps feed CVA, the responsive engine, the safelist, and the TypeScript types; nothing is written twice.
4. **Own the runtime** — no Radix, no Floating UI, no framer-motion in the library; five hooks and ~400 lines replace them. Maximum control, maximum responsibility.
5. **The consumer never sees the machinery** — precompiled CSS, prefixed classes, one import. Tailwind is an implementation detail (§2.3's caveat noted).

Everything in the analysis report — the bugs, the inconsistencies, the competitor gaps — is measured against how well the code keeps these five promises.